DATABASE MANAGEMENT SYSTEM

MINI PROJECT

RESTAURANT MANAGEMENT SYSTEM

A Relational Database Project using MySQL

SUBMITTED BY

Hardeep Singh Sodhi

Class: 25BCD2 A

UID: 25BCD10215

SUBMITTED TO

Harkamal Kaur Mam

1. Introduction

A Restaurant Management System (RMS) is a database-based system used to store, organize, update, and retrieve information related to the daily activities of a restaurant. A restaurant normally deals with many types of information, such as customer details, food items, menu prices, orders, quantities, and sales. If these records are maintained manually in notebooks or separate files, the work can become slow and difficult. Manual records may also contain duplicate entries, calculation mistakes, missing information, and problems while searching for previous orders.

A Database Management System (DBMS) solves many of these problems by storing information in an organized and structured form. In this mini project, MySQL is used as the DBMS and SQL (Structured Query Language) is used to create tables, insert records, retrieve data, update prices, and calculate sales. The project uses a relational database in which information is stored in separate but connected tables. This avoids keeping all information in one large table and makes the data easier to manage.

The database contains three main tables: Customer, Menu, and Orders. The Customer table stores a unique customer ID, customer name, and phone number. The Menu table stores a unique item ID, food item name, and price. The Orders table records each order, the customer who placed it, the menu item ordered, the quantity, and the order date. Primary keys identify records uniquely, while foreign keys connect the order to the relevant customer and menu item.

The project demonstrates common SQL operations, including CREATE DATABASE, CREATE TABLE, INSERT, SELECT, JOIN, UPDATE, and the aggregate function SUM(). For example, a JOIN query combines data from the three tables to display the customer name, food item, quantity, price, and total amount for each order. The SUM() function calculates total sales from the recorded orders. This is a basic academic model; a real restaurant system may also include staff, table reservations, payment methods, inventory, tax, discounts, and online ordering.

2. Objectives / Problem Statement

2.1 Problem Statement

In a small restaurant, information about customers, menu items, and orders may be recorded manually. Searching old orders can take time, changing a menu price requires updating records carefully, and calculating daily sales by hand can cause errors. If the same customer or food item details are written repeatedly, duplicate or inconsistent information may be created. Therefore, a simple database is needed to store related restaurant records in a systematic manner and retrieve useful information quickly.

2.2 Objectives

To design a simple relational database for restaurant operations.

To store customer details such as customer ID, name, and phone number.

To maintain a menu containing food item names and their prices.

To record customer orders, order quantities, and order dates.

To connect tables through primary keys and foreign keys.

To retrieve complete order details using SQL JOIN operations.

To calculate the amount for each order using quantity multiplied by item price.

To calculate total sales using the SUM() aggregate function.

To update menu prices when a restaurant changes its pricing.

To reduce repetitive manual work and demonstrate the practical use of DBMS concepts.

2.3 Scope of the Project

The current project is limited to basic customer, menu, order, and sales information. It is intended for learning database concepts and SQL queries. It does not include a graphical user interface, login system, online payment gateway, stock tracking, or automatic tax calculation. These features can be added in a future version.

3. Modules and Tools / Technologies Used

3.1 Modules

Customer Management Module Stores customer information in the Customer table. Each customer has a unique customer_id, which helps identify the correct customer and link their orders to their details.

Menu Management Module Stores the available food items and their prices in the Menu table. The item_id uniquely identifies each menu item. The UPDATE statement can be used when a food item's price changes.

Order Management Module Stores each order in the Orders table. It records the order ID, customer ID, item ID, quantity, and date. Foreign keys help prevent an order from referring to a customer or menu item that does not exist.

Order Details and Billing Module Combines customer, order, and menu information using JOIN. The expression quantity * price calculates the line amount for each recorded order. In this simplified project, each order record refers to one menu item.

Sales Report Module Uses SUM(quantity * price) to calculate the total value of the sample orders. This helps demonstrate how a restaurant can summarize sales using an SQL aggregate function.

3.2 Tools and Technologies Used

MySQL: A relational database management system used to create and manage the database.

SQL: A language used to define database tables and insert, retrieve, and update records.

MySQL Workbench / MySQL Command Line: An environment in which SQL statements can be executed and results viewed.

Relational Database Model: Stores data in tables connected through keys.

Primary Key: A field that uniquely identifies each record in a table.

Foreign Key: A field that references a key in another table and establishes a relationship.

3.3 Database Table Design

The project uses the following three tables:

Relationship: One customer may place multiple orders, and one menu item may appear in multiple order records. In this simplified design, each row in Orders refers to one customer and one menu item.

4. Code with Outputs

The following SQL statements are written for MySQL. Execute them in the given order: first create the database and tables, then insert sample data, and finally run the queries. The sample records are used to demonstrate how the system works.

4.1 Create Database and Tables

```
CREATE DATABASE restaurant_management;

USE restaurant_management;

CREATE TABLE Customer (

customer_id INT PRIMARY KEY,

customer_name VARCHAR(50) NOT NULL,

phone VARCHAR(15)
```

```
);
CREATE TABLE Menu (
item_id INT PRIMARY KEY,
item_name VARCHAR(50) NOT NULL,
price DECIMAL(8,2) NOT NULL
);
CREATE TABLE Orders (
order_id INT PRIMARY KEY,
customer_id INT,
item_id INT,
quantity INT NOT NULL,
order_date DATE,
FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),
FOREIGN KEY (item_id) REFERENCES Menu(item_id)
);
```

4.2 Insert Sample Data

```
INSERT INTO Customer VALUES
(1, 'Aman', '9876543210'),
(2, 'Simran', '9876501234');
INSERT INTO Menu VALUES
(101, 'Pizza', 250.00),
(102, 'Burger', 150.00),
(103, 'Pasta', 200.00);
INSERT INTO Orders VALUES
(1001, 1, 101, 2, '2026-10-20'),
(1002, 2, 102, 1, '2026-10-21'),
(1003, 1, 103, 1, '2026-10-21');
```

4.3 Display the Menu

```
SELECT * FROM Menu;
```

4.4 Display Order Details

```
SELECT O.order_id, C.customer_name, M.item_name,
O.quantity, M.price,
(O.quantity * M.price) AS total_amount,
O.order_date
FROM Orders O
JOIN Customer C ON O.customer_id = C.customer_id
JOIN Menu M ON O.item_id = M.item_id;
```

4.5 Calculate Total Sales

```
SELECT SUM(O.quantity * M.price) AS total_sales
FROM Orders O
JOIN Menu M ON O.item_id = M.item_id;
```

4.6 Update a Menu Price

```
UPDATE Menu
SET price = 160.00
```

```
WHERE item_id = 102;

SELECT * FROM Menu WHERE item_id = 102;
```

4.7 Explanation of Important SQL Commands

CREATE DATABASE: Creates a new database named restaurant_management.

CREATE TABLE: Defines a table and its columns, data types, and constraints.

PRIMARY KEY: Ensures that each record has a unique identifier.

FOREIGN KEY: Connects Orders to the Customer and Menu tables.

INSERT INTO: Adds sample records to a table.

SELECT: Retrieves data from one or more tables.

JOIN: Combines related records from different tables.

SUM(): Adds numeric values to produce a total.

UPDATE: Changes an existing value, such as a menu price.

4.8 Sample Output Screenshots

The following images show the expected results for the sample data. They are illustrative console-style outputs. For a final submission requiring actual screenshots, execute the queries in MySQL Workbench and replace these sample images with screenshots from your own session.

Sample Output 1 — Menu Table

Sample Output 2 — Order Details

Sample Output 3 — Total Sales

Sample Output 4 — Updated Burger Price

5. Conclusion

The Restaurant Management System mini project demonstrates the practical use of a Database Management System in organizing restaurant information. It stores customer details, food menu items, and order records in separate relational tables. Primary keys provide unique identification, and foreign keys maintain relationships between related records. SQL queries are used to insert data, display menu items, combine order information using JOIN, update a menu price, and calculate total sales using SUM().

This project shows that a well-structured database can make information easier to retrieve and update than manually maintained records. It also helps students understand relational tables, keys, constraints, and basic SQL operations through a real-world example. In the future, the project can be extended with employee management, table booking, inventory tracking, billing, discounts, payment details, and a user-friendly interface.

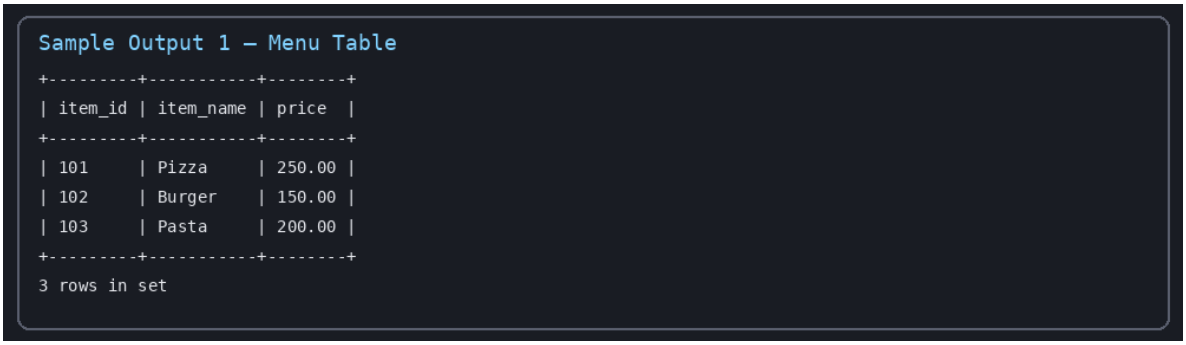
References

MySQL Reference Manual — SQL statements and relational database concepts.

DBMS class notes and lecture material.

SQL practice queries prepared for this academic mini project.

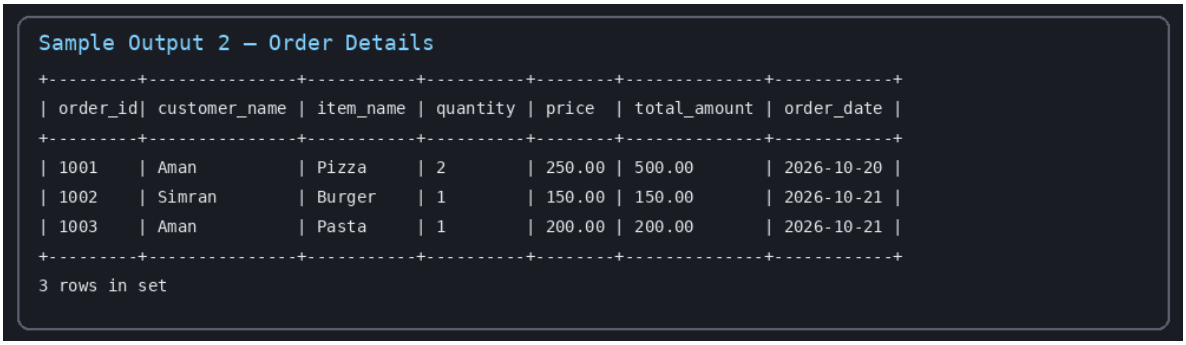
Sample Output Screenshot 1



Sample Output 1 — Menu Table

```
+---------+-----------+--------+
| item_id | item_name | price  |
+---------+-----------+--------+
| 101     | Pizza     | 250.00 |
| 102     | Burger    | 150.00 |
| 103     | Pasta     | 200.00 |
+---------+-----------+--------+
3 rows in set
```

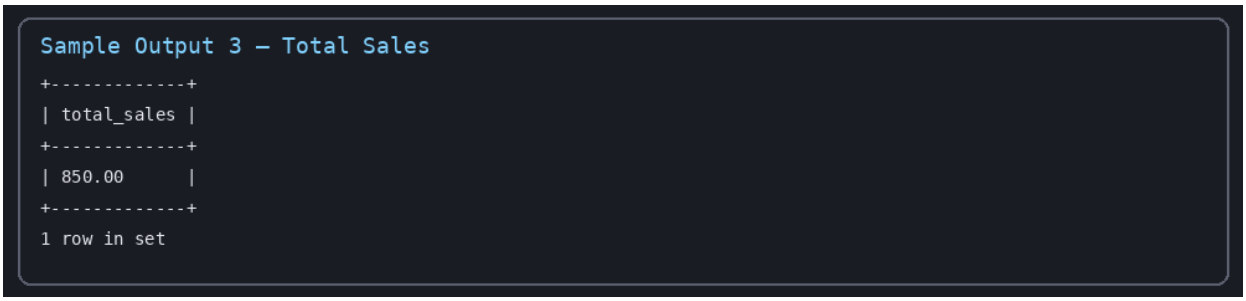
Sample Output Screenshot 2



Sample Output 2 — Order Details

```
+---------+---------------+-----------+----------+--------+--------------+------------+
| order_id| customer_name | item_name | quantity | price  | total_amount | order_date |
+---------+---------------+-----------+----------+--------+--------------+------------+
| 1001    | Aman          | Pizza     | 2        | 250.00 | 500.00       | 2026-10-20 |
| 1002    | Simran        | Burger    | 1        | 150.00 | 150.00       | 2026-10-21 |
| 1003    | Aman          | Pasta     | 1        | 200.00 | 200.00       | 2026-10-21 |
+---------+---------------+-----------+----------+--------+--------------+------------+
3 rows in set
```

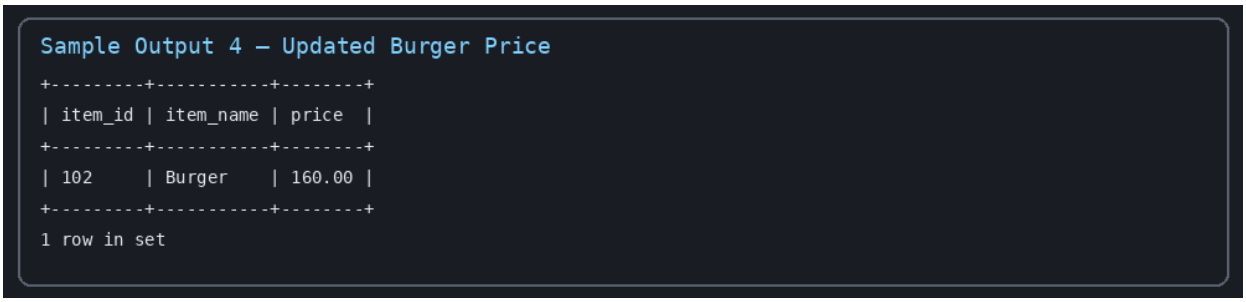
Sample Output Screenshot 3



Sample Output 3 — Total Sales

```
+-------------+
| total_sales |
+-------------+
| 850.00      |
+-------------+
1 row in set
```

Sample Output Screenshot 4



Sample Output 4 — Updated Burger Price

```
+---------+-----------+--------+
| item_id | item_name | price  |
+---------+-----------+--------+
| 102     | Burger    | 160.00 |
+---------+-----------+--------+
1 row in set
```